

HTML5, CSS & JavaScript

Adrian Streitz, Johannes Rank, Borys Levkovskyi



Prerequisites

- Fundamental programming skills
- Successful completion of the chapter *Introduction to SAP Application Development*

Agenda

I. HTML5

II. CSS

III. JavaScript & jQuery

IV. Examples

Fundamentals

- HTML5 = Hypertext Markup Language, Version 5
- No programming language, but a markup language
- Main purpose: Structuring and formatting of content (e.g. texts, pictures, hyperlinks)
- Mainly used for the representation of web documents and web content
- HTML pages are individually rendered by the browser
- HTML5 provides new functionalities compared with HTML4, e.g.
 - New elements for mobile web pages
 - Integration of video and audio contents
 - New graphical elements

Basic structure

- Document type declaration

```
<!DOCTYPE html>
<html>
...
</html>
```

Each HTML page
is enclosed by
the <html>-tags

- Header section: declaration of meta information, page title or links to Java Script or CSS files

Character
encoding for the
browser

```
<head>
<meta charset="UTF-8">
<title>My first HTML5 page</title>
</head>
```

Title of the page

- Body: contains the content of the page

```
<body>
<p>This is my first HTML5 page</p>
</body>
```

Document content,
represented by an
opening tag, the
actual content, and a
closing tag

Text elements and comments

- HTML5 offers a huge variety of text elements to structure text content
- Comments are written in between `<!--` and `-->` (also when used over several lines)

This is a headline

This is a smaller headline

This is a standard text, followed by a linebreak,
bold words, and a [empty hyperlink](#)

1. This is a first list entry
2. This is a second list entry

-
- This is a first list entry
 - This is a second list entry

```
<body>
  <h1>This is a headline</h1>
  <h2>This is a smaller headline</h2>
  <div id="part1">
    <p>
      <!--This is a comment. Comments are not displayed in the browser-->
      This is a standard text, followed by a linebreak, <br>
      <b>bold words,</b> and a <a href="">empty hyperlink</a>
    </p>
    <ol>
      <!--
      You may write comments also over several text lines,
      as shown in this example.
      -->
      <li>This is a first list entry</li>
      <li>This is a second list entry</li>
    </ol>
  </div>
  <hr>
  <div id="part2">
    <ul>
      <li>This is a first list entry</li>
      <li>This is a second list entry</li>
    </ul>
  </div>
</body>
```

Tables

- Tables are an important element of HTML5 to show a higher amount of data within one view
- It is separated between header data and body data

Head column 1	Head column 2	Head column 3
Body line 1	Body line 1	Body line 1
Body line 2	Body line 2	Body line 2
Body line 3	Body line 3	Body line 3
Body line 4	Body line 4	Body line 4
Body line 5	Body line 5	Body line 5

```
<body>
  <table border='1'>
    <thead>
      <tr>
        <th>Head column 1</th>
        <th>Head column 2</th>
        <th>Head column 3</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Body line 1</td>
        <td>Body line 1</td>
        <td>Body line 1</td>
      </tr>
      <tr>
        <td>Body line 2</td>
        <td>Body line 2</td>
        <td>Body line 2</td>
      </tr>
      ...
    </tbody>
  </table>
</body>
```

I. HTML5

Forms

- Forms are used for interaction with the user
- Typical forms are input screens, which can be processed e.g. with JavaScript

Timesheet

Name

Task

Hours

Description

Billable ☐

```
<body>
  <form method="get" action="http://targetserver">
    <fieldset id="timesheet">
      <legend>Timesheet</legend>
      <label for="name">Name</label> <input type="text" name="name" /> <br />
      <label for="task">Task</label> <select name="task" size="1">
        <option value="consulting">Consulting</option>
        <option value="development">Development</option>
      </select> <br />
      <label for="hours">Hours</label> <input type="number"
        name="Hours" min="0" max="8" id="Hours" /> <br />
      <label for="desc">Description</label>
      <textarea name="desc" rows="3"></textarea>
      <br />
      <label for="billing">Billable</label> <input type="checkbox"
        name="billing" id="BILL"> <br />
      <input type="submit"
        value="Submit" />
    </fieldset>
  </form>
</body>
```

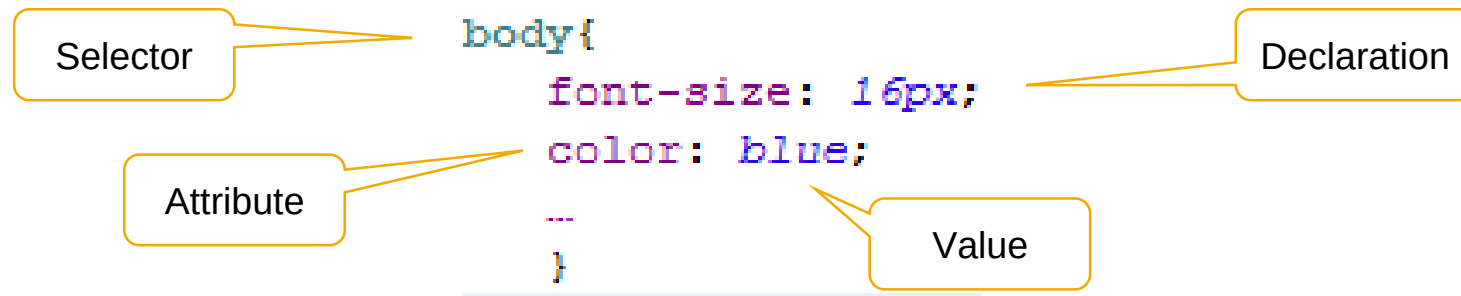

Now you know the fundamentals of HTML5.

To consolidate your knowledge, you can do task 1 and 2 of the HTML5, CSS & JavaScript exercise.

II. CSS

Fundamentals

- CSS = Cascading Style Sheets
- CSS is used to format the content structured by HTML
- Designing and positioning of the elements
- CSS contains information about the position, the size and type of the text, background color, etc.
- Syntax consists of selector and declaration



Integrating CSS into HTML

- There are three possibilities to integrate CSS into HTML
- Style definitions are evaluated in a hierarchical order

1. Separate File

2. Direct
declaration in
header section

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>CSS example</title>

<link href="style.css" rel="stylesheet" />

<style type="text/css">
label {
    width: 100px;
    float: left;
    background-color: silver;
}
</style>
</head>
<body>
    <label for="name" style="background-color: yellow;">Name</label>
    <input type="text" name="name" />
    <br />

</body>
</html>
```

3. Inline
declaration

II. CSS

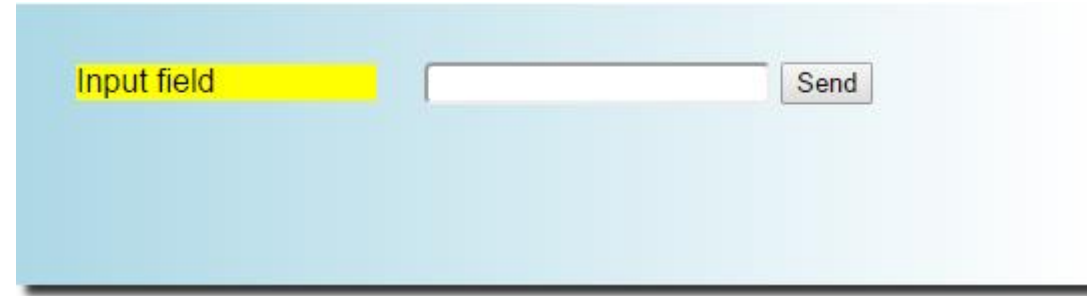
Selectors

With the selector you define the HTML elements for which the format definition is valid for:

- Universal selector: valid for all elements
`*{attribute: value;}`
- Type selector: valid for specific elements
`label {attribute: value;}`
- ID selector: valid for all elements with the respective ID
`#main {attribute: value;}`
- Class selector: valid for all elements with the respective class attribute
`.Navigation {attribute: value;}`
- Descendant selector: valid for elements if they are part of a specific other element
`p i {attribute: value;}`

II. CSS

HTML5 and CSS combined



```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>HTML5 & CSS Demo</title>
<link href="style.css" rel="stylesheet" />
<style>
input {
  margin-left: 10px;
  border-radius: 4px;
}
</style>
</head>
<body>
<div id="Site">
  <div id="Form">
    <label style="background-color: yellow">Input field</label> <input
      type="text" />
    <button class="button" type="button">Send</button>
  </div>
</div>
</body>
</html>
```

Valid for all
input fields

Overwrites
style definition
from label

```
* {
  font-family: Arial, Helvetica, sans-serif;
}

label {
  background-color: red;
  margin: 10px;
  display: inline-block;
  width: 150px;
  margin: 10px;
}

#Site {
  background-image: -webkit-linear-gradient(left, lightblue, white);
  -webkit-box-shadow: 5px 5px 5px #333333;
  padding: 20px;
  height: 100px;
  width: 500px;
}
```

Valid for all
elements

Valid for all
labels

Valid for all
elements
with ID "Site"

Now you know the fundamentals of CSS.

To consolidate your knowledge, you can do task 3 of the HTML5, CSS & JavaScript exercise.

III. JavaScript & jQuery

Fundamentals

- Script language for dynamic HTML pages in web browsers
- Determines behavior of web pages
- JavaScript files are not executable, but valid when integrated into HTML
- Remaining problem: different interpretation by web browsers
- Basis for many frameworks and libraries (e.g. jQuery, SAPUI5)
- Knowing JavaScript is fundamental to understand the dynamic behavior of web pages

Integrating JavaScript into HTML

Three possibilities to integrate JavaScript into HTML:

As a separate file →
refer to it in the head

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="ISO-8859-1">
  <script src='script.js' type='text/javascript'></script>
  <title>Insert title here</title>
  <script type="text/javascript">
    function myFunction() {
      //Some Code
    }
  </script>
</head>
<body>
  <p>This is my first HTML5 page</p>
  <script type="text/javascript">
    function myFunction() {
      //Some Code
    }
  </script>
</body>
</html>
```

Within the head

Within the body

III. JavaScript & jQuery

Variables

- Dynamically typed

```
var sVar = 'text';      // string
var iVar = 123;         // integer
var fVar = 12.34;       // float
var oVar = new Object(); // object
```

- Variables defined outside of a function are global
- Omitting the keyword **var** inside a function makes the variable global

III. JavaScript & jQuery

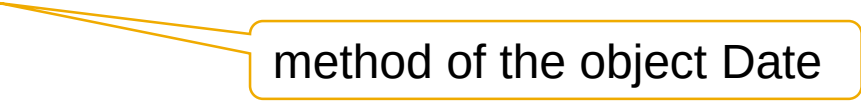
Objects

- Creating an object:

```
var object1 = new Object();  
var object1{  
  attribute1:'value1';  
  ...  
  method: function(){  
    ...  
  };  
};
```

- Predefined objects: e.g. Date, Math, String, Array, etc.
- Using the methods of an object

```
var date1 = new Date();  
var date2 = date1.getDate();
```



method of the object Date

III. JavaScript & jQuery

JSON Format

Usage

- Transmit data objects in a human-readable fashion
- Language-independent format (although derived from javascript objects)

DATA-Types

- String
- Number
- boolean
- (JSON) object
- Array
- Null

Example

```
{
  "attribute": "value",
  "attribute": "value",
  "attribute": ["value", "value", ...],
  "attribute": {"attribute": "value", ...}
}

{
  "name": "schmitt",
  "country": "germany",
  „age": 56,
  "height": 1.87,
  "married": true,
  "children": null,
  "hobbies": ["soccer", "tennis", ...],
  "car": {"brand": "BMW", "seats": 4}
}
```

Array, [value,...]

Object

III. JavaScript & jQuery

Functions

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset='UTF-8'>
    <script type='text/javascript'>
      function button() {
        //call function
        alert ('Press button. ');
      }
    </script>
  </head>
  <body>
    //...
  </body>
</html>
```

keyword 'function'

function name

curly brackets

Anonymous functions

- Define function without naming it
(e.g. when it's only used once, or is called right after it has been defined)
- Commonly used when function is immediately executed in program flow or passed as a parameter (especially in event handling)

```
(function(a,b)
{alert(a+b);})(1,2);
//alerts 3
```

Used for in-place
call

```
var a = 1;
var b = 2;
var sum = (function() {
    return a + b;
})();
//sum = 3
```

Used for variable
definition

III. JavaScript & jQuery

Control structures

- If:

```
if (condition){
    //true
} else{
    //false
};
```
 - Switch:

```
switch(condition){
    case value1: [code];
        break;
    case value2: [code];
        break;
    default: [code];
};
```
 - For:

```
for([variable]; [condition]; [operator]){
    //loop body
};
```
 - While:

```
while ([condition]){
    //loop body
};
```
- ```
if ((age >= 18) && (age < 60)){
 ...
}

var oName = document.getElementById('day');
switch(oName.value){
 case 'Monatg': alert ('I do not like Mondays');
 break;
 ...
 default: alert ('No Monday!');
};

for (i=0; i<10; i++){
 i++;
};

while(i<10){
 i++;
};
```



## Source code comments

Multi-line  
comments using `/*`  
and `*/`

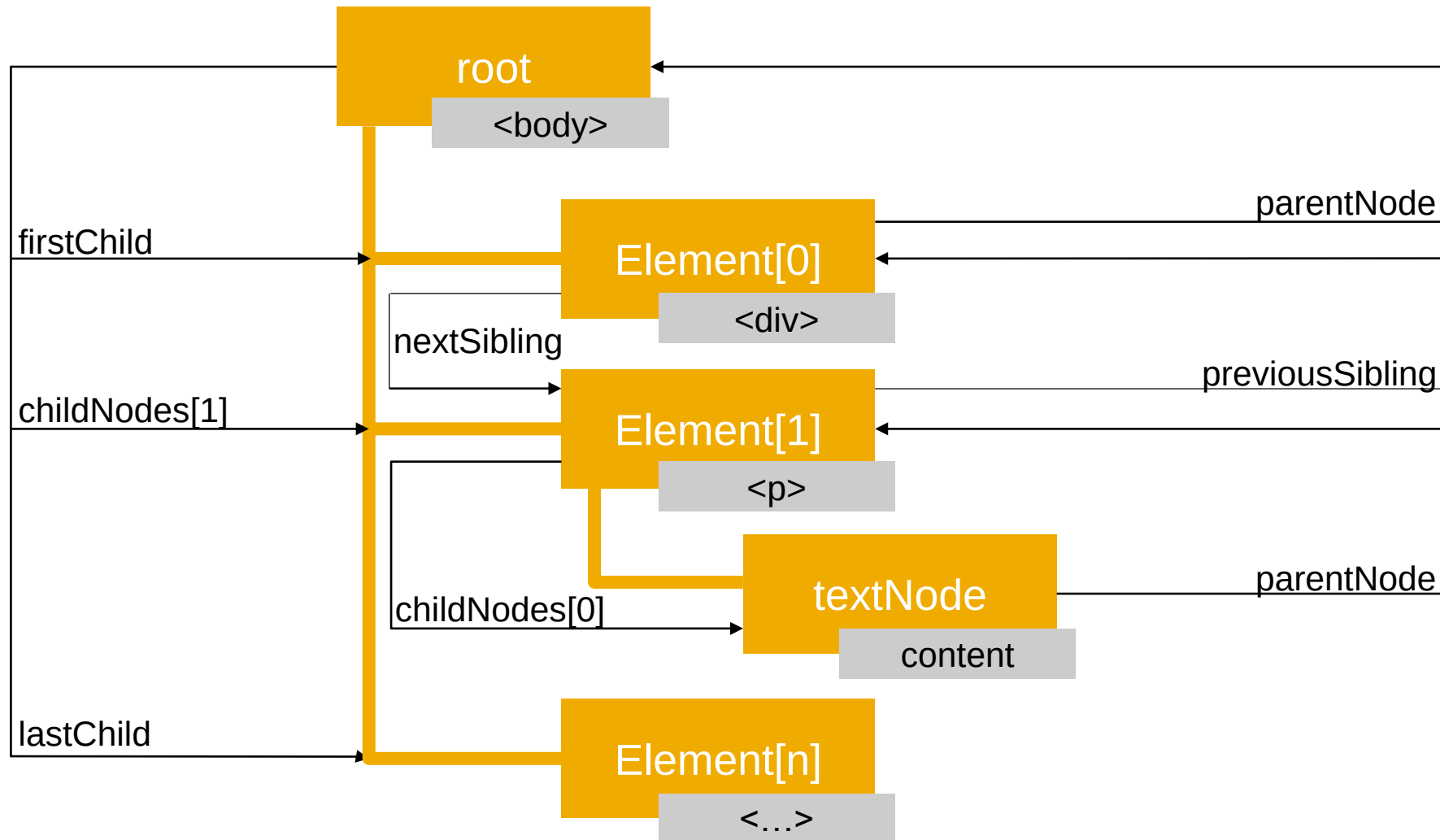
```
<script type="text/javascript">
 function button() {
 //This is a single line JavaScript comment
 alert('Press button.');
```

```
 /*
 This is a multi-line comment
 which may be used for longer
 comments and more detailed explanations.
 */
 }
</script>
```

single line  
comment with `//`

## Document Object Model (DOM)



## JavaScript event handling

- Event Handler: instructions that are executed when an event occurs (e.g. user clicks a button)
- Event-handling copes with non-deterministic incidents (uncertainty what user will do, or how long it takes the page to load) without stopping the program flow (like in the case of polling)

```
<button onclick="(function(){alert('Hello');})();">
Submit</button>
```

Assign anonymous  
function as action

Anonymous function  
as parameter for  
click event

```
document.getElementById('button1').addEventListener('click',function()
{
 alert('Hello');
});
```

### III. JavaScript & jQuery

## JavaScript events

---

- `onabort`: **cancellation**
- `onblur`: **quitting**
- `onchange`: **change**
- `onclick`: **click**
- `ondblclick`: **double click**
- `onerror`: **error**
- `onfocus`: **activation**
- `onkeydown`: **pressing key**
- `onkeypress`: **pressing key**
- `onkeyup`: **letting the key go**
- `onload`: **loading a file**
- `onmousedown`: **pressing mouse**
- `onmousemove`: **moving mouse**
- `onmouseout`: **letting the element go**
- `onmouseover`: **touching the element**
- `onmouseup`: **letting mouse go**
- `onreset`: **resetting the formular**
- `onselect`: **selecting text**
- `onsubmit`: **submitting the formular**
- `onunload`: **leaving the file**

### III. JavaScript & jQuery

## jQuery selectors

- Factory function `$()`: access to objects
- Ready function: the function is being executed after loading the page

```
<!DOCTYPE html>
<html>
 <head>
 <meta charset='UTF-8'>
 <script type='text/javascript'
 src='jquery.js'>
 </script>
 <script type='text/javascript'>
 $(document).ready(function() {
 //code
 });
 </script>
 </head>
 <body>
 ...
 </body>
</html>
```

CSS	jQuery
element {...}	<code>\$('element');</code>
#id	<code>\$('#id');</code>
.class	<code>\$('.class');</code>

### III. JavaScript & jQuery

## jQuery events

---

- `.bind`: binding the handler to the event
- `.on`: binding the handler to the event
- `.blur`: when an element loses the focus
- `.click`: click
- `.dblclick`: double click
- `.hover`: moving the mouse pointer over an element
- `.mousemove`: moving the mouse pointer inside an element
- `.keypress`: pressing a key on the keyboard
- `.keyup`: letting the key go
- `.change`: change of the formular field

## jQuery DOM manipulation

---

- `.after()` : add the content behind the element
- `.append()` : add the content at the end of the element
- `.appendTo()` : add the content at the end of the element
- `.insertAfter()` : attach the content to the element
- `.insertBefore()` : attach the content before the element
- `.prepend()` : attach the content at the beginning of the element
- `.prependTo()` : attach the content at the beginning of the element
- `.remove()` : remove the element
- `.wrap()` : use the HTML structure
- `.addClass()` : add the class to the elements
- `.removeClass()` : remove the class
- `.css()` : set the css attribute

Now you know the fundamentals of JavaScript and jQuery.

To consolidate your knowledge, you can do task 4 of the HTML5, CSS & JavaScript exercise.



# Check your knowledge

# Check your knowledge

---

- A web browser with internet connection is needed to show a rendered HTML5 page
  - ☐ True   ☐ False
- Document type declaration, header section and body are completely separated in HTML5
  - ☐ True   ☐ False
- Compare text elements, tables, and forms. For which purpose should they be mainly used?
- If all three integration possibilities for CSS define a different background color for the same element, which style definition is dominant?
  - ☐ Declaration in separate file
  - ☐ Direct declaration in header section
  - ☐ Inline declaration
- Why is it preferable to use jQuery instead of native JavaScript?

# Solution

# Solution

---

- A web browser with internet connection is needed to show a rendered HTML5 page  
☐ True   ☒ False
- Document type declaration, header section and body are completely separated in HTML5  
☒ True   ☐ False
- Compare text elements, tables, and forms. For which purpose should they be mainly used?  
See section *HTML5*
- If all three integration possibilities for CSS define a different background color for the same element, which style definition is dominant?  
☐ Declaration in separate file  
☐ Direct declaration in header section  
☒ Inline declaration
- Why is it preferable to use jQuery instead of native JavaScript?  
See section *JavaScript & jQuery*

# References

---

- Antolovic, M.: (2016): Einführung in SAPUI5, SAP PRESS 2016

# © 2018 SAP SE or an SAP affiliate company. All rights reserved.

---

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. Please see <http://global12.sap.com/corporate-en/legal/copyright/index.epx> for additional trademark information and notices.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors.

National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP SE or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP SE or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

In particular, SAP SE or its affiliated companies have no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, and SAP SE's or its affiliated companies' strategy and possible future developments, products, and/or platform directions and functionality are all subject to change and may be changed by SAP SE or its affiliated companies at any time for any reason without notice. The information in this document is not a commitment, promise, or legal obligation to deliver any material, code, or functionality. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Readers are cautioned not to place undue reliance on these forward-looking statements, which speak only as of their dates, and they should not be relied upon in making purchasing decisions.